STAT534 Hw7

Problem 1: Sample Covariance Matrix

Part A: My Solution

- I create `makeCovariance(gsl_matrix *covX, gsl_matrix *X)` to compute the sample covariance matrix of the data matrix.
- My solution uses GSL's `gsl_stats_variance` and `gsl_stats_covariance` for easier handling of the matrix objects.
- I accelerate the process by using the symmetric property of covariance matrix: `cov(j, k) = cov(k, j)`, reducing redundant computations.
- Output is saved to `sigma.txt` instead of printing the full matrix directly.

```
void makeCovariance(gsl_matrix *covX, gsl_matrix *X)
{
    size_t n = X->size1;
    size_t p = X->size2;

    // since covariance matrix is symmetric, (j, k) = (k, j), for j != k
    // diagonal case
    for (size_t j = 0; j < p; j++)
    {
        double cov_jj = gsl_stats_variance(gsl_matrix_const_ptr(X, 0, j),
X->tda, n);
        gsl_matrix_set(covX, j, j, cov_jj);
    }
    // off-diagonal
    for (size_t j = 0; j < p; j++)
    {
        for (size_t k = j + 1; k < p; k++)
        {
            double cov_jk = gsl_stats_covariance(
                gsl_matrix_const_ptr(X, 0, j), X->tda,
                gsl_matrix_const_ptr(X, 0, k), X->tda,
                n);
            gsl_matrix_set(covX, j, k, cov_jk);
            gsl_matrix_set(covX, k, j, cov_jk);
        }
    }
}
```

Part B: AI Solution

Prompt

Read Problem 1 from `hw7.pdf`. Write C++ code for the function:

```
void makeCovariance(gsl_matrix* covX, gsl_matrix* X)
```

The function should store the sample covariance matrix of data matrix `X` into `covX`. Use GSL matrix objects. Save the AI code into `ai_matrices.cpp`.

Code Generated

```
void ai_makeCovariance(gsl_matrix *covX, gsl_matrix *X)
{
    size_t n = X->size1;
    size_t p = X->size2;

    if (covX->size1 != p || covX->size2 != p)
    {
        fprintf(stderr, "Error: covariance matrix has wrong dimension.\n");
        exit(1);
    }

    double *means = new double[p];

    for (size_t j = 0; j < p; j++)
    {
        double sum = 0.0;
        for (size_t i = 0; i < n; i++)
        {
            sum += gsl_matrix_get(X, i, j);
        }
        means[j] = sum / n;
    }

    for (size_t j = 0; j < p; j++)
    {
        for (size_t k = 0; k < p; k++)
        {
            double cov = 0.0;
            for (size_t i = 0; i < n; i++)
            {
                cov += (gsl_matrix_get(X, i, j) - means[j]) *
                       (gsl_matrix_get(X, i, k) - means[k]);
            }
            gsl_matrix_set(covX, j, k, cov / (n - 1));
        }
    }

    delete[] means;
}
```

Part C: Comparison

- **Overall**
  - Both solutions calculate the same sample covariance matrix.
  - My solution is more tied to the assignment hint because it uses `gsl_stats_covariance`.
  - AI solution is more from-scratch and explicitly implements the covariance formula.
- **Correctness**
  - Both compute the same sample covariance.
  - AI solution also checks output matrix dimension, which will be more robust.
- **Data Structure**
  - Both use `gsl_matrix*`.
  - AI solution allocates an extra `means` array.
  - My solution does not need extra mean storage because GSL stats library handle it.
- **Readability**
  - AI solution is easier to understand if reading the covariance formula directly.
  - My solution is shorter and cleaner if familiar with GSL.
- **Algorithm**
  - My solution uses the symmetric covariance matrix property and only computes half of the off-diagonal entries.
  - AI solution computes all `(j, k)` entries separately, so it repeats work for `(j, k)` and `(k, j)`.

Problem 2: Cholesky and Multivariate Normal Samples

Part A: My Solution

- I create `makeCholesky` to copy the input covariance matrix first, since GSL Cholesky decomposition modifies the matrix in-place.
- After calling `gsl_linalg_cholesky_decomp`, I manually set upper triangular entries to zero, because later I only want to use the lower triangular matrix `L`. (based on the doc, the upper part should be ignored.)
- For `randomMVN`, I first draw standard normal vector `Z`, then compute `X = LZ`.
- I let `randomMVN` generate all rows in the `samples` matrix, so it works for one draw or many draws.

```
gsl_matrix *makeCholesky(gsl_matrix *K)
{
    gsl_matrix *chol = gsl_matrix_alloc(K->size1, K->size2);
    gsl_matrix_memcpy(chol, K);

    gsl_linalg_cholesky_decomp(chol); // this modifies the input in-place
    // based on the documentation, it will only modify the lower triangular
part
    // need to ignore the upper tri part when doing calculations
afterwards.
```

```
    for (size_t i = 0; i < chol->size1; i++)
    {
        for (size_t j = i + 1; j < chol->size2; j++)
        {
            gsl_matrix_set(chol, i, j, 0.0);
        }
    }

    return chol;
}

void randomMVN(gsl_rng *mystream, gsl_matrix *samples, gsl_matrix *sigma)
{
    size_t p = sigma->size1;
    gsl_matrix *chol = makeCholesky(sigma);
    gsl_matrix *Z = gsl_matrix_alloc(p, 1);
    gsl_matrix *X = gsl_matrix_alloc(p, 1);

    for (size_t i = 0; i < samples->size1; i++)
    {
        draw_multivariate_standard_normal(mystream, Z);
        gsl_blas_dgemm(CblasNoTrans, CblasNoTrans, 1.0, chol, Z, 0.0, X);

        for (size_t j = 0; j < samples->size2; j++)
        {
            gsl_matrix_set(samples, i, j, gsl_matrix_get(X, j, 0));
        }
    }

    gsl_matrix_free(chol);
    gsl_matrix_free(Z);
    gsl_matrix_free(X);
}
```

Part B: AI Solution

Prompt

Read Problem 2 from `hw7.pdf`. Write C++ code for:

```
gsl_matrix* makeCholesky(gsl_matrix* K);
void randomMVN(gsl_rng* mystream, gsl_matrix* samples, gsl_matrix* sigma);
```

The Cholesky function should return a lower triangular matrix. The randomMVN function should draw multivariate normal samples with covariance matrix `sigma` using the Cholesky method. Put the AI code in `ai_matrices.cpp`.

Code Generated

```
gsl_matrix *ai_makeCholesky(gsl_matrix *K)
{
    if (K->size1 != K->size2)
    {
        fprintf(stderr, "Error: Cholesky decomposition needs a square
matrix.\n");
        exit(1);
    }

    gsl_matrix *L = gsl_matrix_alloc(K->size1, K->size2);
    gsl_matrix_memcpy(L, K);

    int status = gsl_linalg_cholesky_decomp(L);
    if (status != 0)
    {
        fprintf(stderr, "Error: Cholesky decomposition failed.\n");
        gsl_matrix_free(L);
        exit(1);
    }

    for (size_t i = 0; i < L->size1; i++)
    {
        for (size_t j = i + 1; j < L->size2; j++)
        {
            gsl_matrix_set(L, i, j, 0.0);
        }
    }

    return L;
}

void ai_randomMVN(gsl_rng *mystream, gsl_matrix *samples, gsl_matrix
*sigma)
{
    size_t num_samples = samples->size1;
    size_t p = sigma->size1;

    if (samples->size2 != p || sigma->size2 != p)
    {
        fprintf(stderr, "Error: sample and covariance dimensions do not
match.\n");
        exit(1);
    }

    gsl_matrix *L = ai_makeCholesky(sigma);
    gsl_vector *z = gsl_vector_alloc(p);
    gsl_vector *x = gsl_vector_alloc(p);

    for (size_t i = 0; i < num_samples; i++)
    {
        for (size_t j = 0; j < p; j++)
        {
```

```
            gsl_vector_set(z, j, gsl_ran_ugaussian(mystream));
        }

        gsl_blas_dgemv(CblasNoTrans, 1.0, L, z, 0.0, x);

        for (size_t j = 0; j < p; j++)
        {
            gsl_matrix_set(samples, i, j, gsl_vector_get(x, j));
        }
    }

    gsl_vector_free(z);
    gsl_vector_free(x);
    gsl_matrix_free(L);
}
```

Part C: Comparison

- **Overall**
  - Both solutions use the same Cholesky idea: if `Z ~ N(0, I)`, then `LZ ~ N(0, LL^T)`.
  - AI solution is more robust with dimension and decomposition status checks.
- **Correctness**
  - Both copy `sigma` before Cholesky, so the original covariance matrix is not changed.
  - Both clear the upper triangular part after GSL decomposition.
- **Data Structure**
  - My solution stores `Z` and `X` as `gsl_matrix` with one column.
  - AI solution stores `z` and `x` as `gsl_vector`, which is a bit more natural for this operation after some thinking.
- **Readability**
  - My version has a separate `draw_multivariate_standard_normal`, which makes the random draw step cleaner.
  - AI version has all logic in one function, which is self-contained but a little longer.
- **Algorithm**
  - We both use GSL BLAS function to compute the matrix multiplication:
    - My solution uses `gsl_blas_dgemm` for matrix-matrix multiplication with `p x 1` matrices.
    - AI solution uses `gsl_blas_dgemv`, which is more semantically correct for multiplying matrix and vector.
  - In this case, both should have very similar runtime because the dimension is small.

Problem 3: Simulated Covariance From 10000 MVN Draws

Part A: My Solution

- I load the original data and compute its covariance matrix `sigma` first.
- Then I set up a GSL random stream with fixed seed `5566`, so the result is reproducible.
- I use `randomMVN` to generate `10,000` samples from `N(0, sigma)`.
- Finally, I call `makeCovariance` again on the simulated samples and save the output to `random-sample-covariance.txt`.
- I also save the raw random samples to `random-sample.txt`, mainly for debugging and checking.

```
int main()
{
    const int n = 158;
    const int p = 51;
    const int num_draws = 10000;

    char datafilename[] = "erdata.txt";
    char sigma_output_file[] = "sigma.txt";
    char random_sample_output_file[] = "random-sample.txt";
    char random_sample_cov_output_file[] = "random-sample-covariance.txt";

    gsl_matrix *data = gsl_matrix_alloc(n, p);
    gsl_matrix *sigma = gsl_matrix_alloc(p, p);
    gsl_matrix *samples = gsl_matrix_alloc(num_draws, p);
    gsl_matrix *sample_cov = gsl_matrix_alloc(p, p);

    load_data(datafilename, data);
    makeCovariance(sigma, data);
    save_matrix(sigma_output_file, sigma);

    const gsl_rng_type *T = gsl_rng_mt19937;
    gsl_rng *rng = gsl_rng_alloc(T);
    gsl_rng_set(rng, 5566);

    randomMVN(rng, samples, sigma);
    save_matrix(random_sample_output_file, samples);

    makeCovariance(sample_cov, samples);
    save_matrix(random_sample_cov_output_file, sample_cov);

    gsl_rng_free(rng);
    gsl_matrix_free(data);
    gsl_matrix_free(sigma);
    gsl_matrix_free(samples);
    gsl_matrix_free(sample_cov);

    return 0;
}
```

Part B: AI Solution

Prompt

Read Problem 3 from `hw7.pdf`. Write C++ code that draws 10000 samples from a multivariate normal distribution with covariance matrix `sigma`, then computes and stores the sample covariance matrix of those draws. Reuse `randomMVN` and `makeCovariance`. Put the AI code in `ai_matrices.cpp`.

Code Generated

```
void ai_makeRandomSampleCovariance(gsl_rng *mystream,
                                   gsl_matrix *sample_cov,
                                   gsl_matrix *sigma,
                                   int num_draws)
{
    gsl_matrix *samples = gsl_matrix_alloc(num_draws, sigma->size1);

    ai_randomMVN(mystream, samples, sigma);
    ai_makeCovariance(sample_cov, samples);

    gsl_matrix_free(samples);
}
```

Part C: Comparison

- **Overall**
  - Both solutions reuse previous functions from Problem 1 and Problem 2.
  - My solution is written directly in `main`, so it is easier to follow in the main file.
  - AI solution wraps the simulation-and-covariance step into one helper function, which I think it's a bit better and modularized.
- **Correctness**
  - Both generate `10000` draws and compute the covariance matrix from those draws.
  - Both produce a simulated covariance matrix close to the original `sigma`.
  - After check (check.cpp), the absolute mean difference is around 0.008, which is sufficiently small.
- **Data Structure**
  - Both use one `samples` matrix with shape `n x p`.
  - Both use a `p x p` matrix to store the sample covariance.
- **Readability**
  - AI solution contains more modularized functions than mine, which is a bit easier to follow when reading in my opinion.
- **Algorithm**

- Both algorithms are the same: Cholesky decomposition, random normal draws, then sample covariance.
- Main difference will still be the problem 1 covariance calculation.